Danmarks
Tekniske
Universitet

Execise 1
(Ethernet Frame Check Sequence)

FPGA design for communication systems(34349)

Author

Jonas Pedersen - s225269

March 4, 2026

Contents

1 Serial implementation **1**

2 Synthesis and Place & route **3**

3 Testbench serial implementation **3**

4 Parallel implementation **5**

5 Synthesis and Place & route **8**

6 Testbench Parallel implmentation **8**

References **11**

7 Appendix **12**

7.1 MATLAB for Generation of CRC Value 12

7.2 MATLAB for Generating the Parallel Setup 13

1 Serial implementation

This chapter walks through the serial implementation of an FCS circuit, as described in the introduction. The implemented code is shown below.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fcs_check_serial is
  port (
    clk            : in std_logic;
    reset          : in std_logic;
    start_of_frame : in std_logic;
    end_of_frame   : in std_logic;
    data_in        : in std_logic;
    fcs_error      : out std_logic
  );
end fcs_check_serial;

architecture behavioral of fcs_check_serial is
  signal Crc_Reg   : std_logic_vector(31 downto 0);
  signal Bit_Count : unsigned(4 downto 0);
  signal Data_Inv  : std_logic;
  signal Fcs_Active : std_logic;
begin

process(clk, start_of_frame, end_of_frame, data_in)
begin
  if(start_of_frame = '1' or end_of_frame = '1' or Bit_Count < 31)
   then
    Data_Inv <= not data_in;
  else
    Data_Inv <= data_in;
  end if;
end process;

process(clk, reset)
begin
  if (reset = '1') then
    Crc_Reg    <= (others => '0');
    Bit_Count  <= (others => '0');
    fcs_error  <= 'X';
    Fcs_Active <= '0';
  elsif rising_edge(clk) then
    if (end_of_frame = '1') then
      Fcs_Active <= '1';
```

```
    end if;

    if (start_of_frame = '1' or end_of_frame = '1') then
      Bit_Count <= (others => '0');
    elsif(Bit_Count < 31) then
      Bit_Count <= Bit_Count + 1;
    end if;

    Crc_Reg(0)  <= Data_Inv xor Crc_Reg(31);
    Crc_Reg(1)  <= Crc_Reg(0)  xor Crc_Reg(31);
    Crc_Reg(2)  <= Crc_Reg(1)  xor Crc_Reg(31);
    Crc_Reg(3)  <= Crc_Reg(2);
    Crc_Reg(4)  <= Crc_Reg(3)  xor Crc_Reg(31);
    Crc_Reg(5)  <= Crc_Reg(4)  xor Crc_Reg(31);
    Crc_Reg(6)  <= Crc_Reg(5);
    Crc_Reg(7)  <= Crc_Reg(6)  xor Crc_Reg(31);
    Crc_Reg(8)  <= Crc_Reg(7)  xor Crc_Reg(31);
    Crc_Reg(9)  <= Crc_Reg(8);
    Crc_Reg(10) <= Crc_Reg(9)  xor Crc_Reg(31);
    Crc_Reg(11) <= Crc_Reg(10) xor Crc_Reg(31);
    Crc_Reg(12) <= Crc_Reg(11) xor Crc_Reg(31);
    Crc_Reg(13) <= Crc_Reg(12);
    Crc_Reg(14) <= Crc_Reg(13);
    Crc_Reg(15) <= Crc_Reg(14);
    Crc_Reg(16) <= Crc_Reg(15) xor Crc_Reg(31);
    Crc_Reg(17) <= Crc_Reg(16);
    Crc_Reg(18) <= Crc_Reg(17);
    Crc_Reg(19) <= Crc_Reg(18);
    Crc_Reg(20) <= Crc_Reg(19);
    Crc_Reg(21) <= Crc_Reg(20);
    Crc_Reg(22) <= Crc_Reg(21) xor Crc_Reg(31);
    Crc_Reg(23) <= Crc_Reg(22) xor Crc_Reg(31);
    Crc_Reg(24) <= Crc_Reg(23);
    Crc_Reg(25) <= Crc_Reg(24);
    Crc_Reg(26) <= Crc_Reg(25) xor Crc_Reg(31);
    Crc_Reg(27) <= Crc_Reg(26);
    Crc_Reg(28) <= Crc_Reg(27);
    Crc_Reg(29) <= Crc_Reg(28);
    Crc_Reg(30) <= Crc_Reg(29);
    Crc_Reg(31) <= Crc_Reg(30);

    if (Fcs_Active = '1' and Bit_Count = 31) then
      if (Crc_Reg = X"00000000") then
        fcs_error <= '0';
      else
        fcs_error <= '1';
```

```
      end if;
      Fcs_Active <= '0';
    end if;
  end if;
end process;

end behavioral;
```

Listing 1: Serial check module

2 Synthesis and Place & route

Synthesis and place & route were performed, determining a worst-case operating frequency of 383.6 MHz at 1200 mV and 85°C.

3 Testbench serial implementation

The testbench below

```
library ieee;
use ieee.std_logic_1164.all;

entity fcs_check_serial_tb is
end fcs_check_serial_tb;

architecture behavior of fcs_check_serial_tb is
  component fcs_check_serial
    port(clk            : in  std_logic;
         reset          : in  std_logic;
         start_of_frame : in  std_logic;
         end_of_frame   : in  std_logic;
         data_in        : in  std_logic;
         fcs_error      : out std_logic);
  end component;

  -- Signals
  signal Test_clk            : std_logic := '0';
  signal Test_reset          : std_logic := '0';
  signal Test_start_of_frame : std_logic := '0';
  signal Test_end_of_frame   : std_logic := '0';
  signal Test_data_in        : std_logic := '0';
  signal Test_fcs_error      : std_logic;

  constant clk_period : time := 1 ns;

```

```
  constant Ethernet_packet : std_logic_vector(511 downto 0) :=
    x"00_10_A4_7B_EA_80_00_12_34_56_78_90_08_00_45" &
    x"00_00_2E_B3_FE_00_00_80_11_05_40_C0_A8_00_2C" &
    x"C0_A8_00_04_04_00_04_00_00_1A_2D_E8_00_01_02" &
    x"03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F_10_11" &
    x"E6_C5_3D_B2";

begin
  Comp1 : fcs_check_serial port map (
    clk            => Test_clk,
    reset          => Test_reset,
    start_of_frame => Test_start_of_frame,
    end_of_frame   => Test_end_of_frame,
    data_in        => Test_data_in,
    fcs_error      => Test_fcs_error
  );

  Clk_Generator : process
  begin
    Test_clk <= '0';
    wait for clk_period / 2;
    Test_clk <= '1';
    wait for clk_period / 2;
  end process;

  Data_Sim : process
  begin
    Test_reset <= '1';
    wait for clk_period;
    Test_reset <= '0';

    for i in Ethernet_packet'range loop
      if i = (Ethernet_packet'length - 1) then
        Test_start_of_frame <= '1';
      else
        Test_start_of_frame <= '0';
      end if;

      Test_data_in <= Ethernet_packet(i);

      if i = 31 then
        Test_end_of_frame <= '1';
      else
        Test_end_of_frame <= '0';
      end if;

```

```
      wait for clk_period;
    end loop;

    Test_data_in <= '0';
    wait;
  end process;

end behavior;
```

Listing 2: Serial FCS Check Testbench

4 Parallel implementation

This presents the parallel implementation of an FCS circuit. The logic assigned to each register is derived from the calculations found in Appendix ??.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fcs_check_parallel is
  port (
    clk            : in std_logic;
    reset          : in std_logic;
    start_of_frame : in std_logic;
    end_of_frame   : in std_logic;
    data_in        : in std_logic_vector(7 downto 0);
    fcs_error      : out std_logic
  );
end fcs_check_parallel;

architecture behavioral of fcs_check_parallel is
  signal Crc_Reg    : std_logic_vector(31 downto 0);
  signal Bit_Count  : unsigned(1 downto 0);
  signal Data_Inv   : std_logic_vector(7 downto 0);
  signal Fcs_Active : std_logic;

begin

  process(start_of_frame, end_of_frame, data_in, Bit_Count)
  begin
    if(start_of_frame = '1' or end_of_frame = '1' or Bit_Count < 3)
   then
      Data_Inv <= not data_in;
    else
      Data_Inv <= data_in;
```

```
    end if;
  end process;

  process(clk, reset)
  begin
    if (reset = '1') then
      Crc_Reg    <= (others => '0');
      Bit_Count  <= (others => '0');
      fcs_error  <= '1';
      Fcs_Active <= '0';

    elsif rising_edge(clk) then
      if (end_of_frame = '1') then
        Fcs_Active <= '1';
      end if;

      if (start_of_frame = '1' or end_of_frame = '1') then
        Bit_Count <= (others => '0');
      elsif(Bit_Count < 3) then
        Bit_Count <= Bit_Count + 1;
      end if;

      Crc_Reg(0)  <= Crc_Reg(24) xor Crc_Reg(30) xor Data_Inv(0);
      Crc_Reg(1)  <= Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(30) xor Crc_Reg(31) xor Data_Inv(1);
      Crc_Reg(2)  <= Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(30) xor Crc_Reg(31) xor Data_Inv(2);
      Crc_Reg(3)  <= Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(31) xor Data_Inv(3);
      Crc_Reg(4)  <= Crc_Reg(24) xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(28) xor Crc_Reg(30) xor Data_Inv(4);
      Crc_Reg(5)  <= Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(27) xor Crc_Reg(28) xor Crc_Reg(29) xor Crc_Reg(30) xor Crc_Reg(31) xor Data_Inv(5);
      Crc_Reg(6)  <= Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(28) xor Crc_Reg(29) xor Crc_Reg(30) xor Crc_Reg(31) xor Data_Inv(6);
      Crc_Reg(7)  <= Crc_Reg(24) xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(29) xor Crc_Reg(31) xor Data_Inv(7);
      Crc_Reg(8)  <= Crc_Reg(0)  xor Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(27) xor Crc_Reg(28);
      Crc_Reg(9)  <= Crc_Reg(1)  xor Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(28) xor Crc_Reg(29);
      Crc_Reg(10) <= Crc_Reg(2)  xor Crc_Reg(24) xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(29);
      Crc_Reg(11) <= Crc_Reg(3)  xor Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(27) xor Crc_Reg(28);
```

```
      Crc_Reg(12) <= Crc_Reg(4)  xor Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(28) xor Crc_Reg(29) xor Crc_Reg(30);
      Crc_Reg(13) <= Crc_Reg(5)  xor Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(29) xor Crc_Reg(30) xor Crc_Reg(31);
      Crc_Reg(14) <= Crc_Reg(6)  xor Crc_Reg(26) xor Crc_Reg(27) xor Crc_Reg(28) xor Crc_Reg(30) xor Crc_Reg(31);
      Crc_Reg(15) <= Crc_Reg(7)  xor Crc_Reg(27) xor Crc_Reg(28) xor Crc_Reg(29) xor Crc_Reg(31);
      Crc_Reg(16) <= Crc_Reg(8)  xor Crc_Reg(24) xor Crc_Reg(28) xor Crc_Reg(29);
      Crc_Reg(17) <= Crc_Reg(9)  xor Crc_Reg(25) xor Crc_Reg(29) xor Crc_Reg(30);
      Crc_Reg(18) <= Crc_Reg(10) xor Crc_Reg(26) xor Crc_Reg(30) xor Crc_Reg(31);
      Crc_Reg(19) <= Crc_Reg(11) xor Crc_Reg(27) xor Crc_Reg(31);
      Crc_Reg(20) <= Crc_Reg(12) xor Crc_Reg(28);
      Crc_Reg(21) <= Crc_Reg(13) xor Crc_Reg(29);
      Crc_Reg(22) <= Crc_Reg(14) xor Crc_Reg(24);
      Crc_Reg(23) <= Crc_Reg(15) xor Crc_Reg(24) xor Crc_Reg(25) xor Crc_Reg(30);
      Crc_Reg(24) <= Crc_Reg(16) xor Crc_Reg(25) xor Crc_Reg(26) xor Crc_Reg(31);
      Crc_Reg(25) <= Crc_Reg(17) xor Crc_Reg(26) xor Crc_Reg(27);
      Crc_Reg(26) <= Crc_Reg(18) xor Crc_Reg(24) xor Crc_Reg(27) xor Crc_Reg(28) xor Crc_Reg(30);
      Crc_Reg(27) <= Crc_Reg(19) xor Crc_Reg(25) xor Crc_Reg(28) xor Crc_Reg(29) xor Crc_Reg(31);
      Crc_Reg(28) <= Crc_Reg(20) xor Crc_Reg(26) xor Crc_Reg(29) xor Crc_Reg(30);
      Crc_Reg(29) <= Crc_Reg(21) xor Crc_Reg(27) xor Crc_Reg(30) xor Crc_Reg(31);
      Crc_Reg(30) <= Crc_Reg(22) xor Crc_Reg(28) xor Crc_Reg(31);
      Crc_Reg(31) <= Crc_Reg(23) xor Crc_Reg(29);

      if (Fcs_Active = '1' and Bit_Count = 3) then
        if (Crc_Reg = X"00000000") then
          fcs_error <= '0';
        else
          fcs_error <= '1';
        end if;
        Fcs_Active <= '0';
      end if;

    end if;
  end process;

```

```
end behavioral;
```

Listing 3: Parallel FCS Check

5 Synthesis and Place & route

Synthesis and place & route were performed, determining a worst-case operating frequency of 453.6 MHz at 1200 mV and 85°C.

6 Testbench Parallel implmentation

The testbench below

```
library ieee;
use ieee.std_logic_1164.all;

entity fcs_check_parallel_tb is
end fcs_check_parallel_tb;

architecture behavior of fcs_check_parallel_tb is

  component fcs_check_parallel
    port(clk            : in  std_logic;
         reset          : in  std_logic;
         start_of_frame : in  std_logic;
         end_of_frame   : in  std_logic;
         data_in        : in  std_logic_vector(7 downto 0);
         fcs_error      : out std_logic);
  end component;

  -- Signals
  signal Test_clk            : std_logic := '0';
  signal Test_reset          : std_logic := '0';
  signal Test_start_of_frame : std_logic := '0';
  signal Test_end_of_frame   : std_logic := '0';
  signal Test_data_in        : std_logic_vector(7 downto 0) := x"00";
  signal Test_fcs_error      : std_logic;

  constant clk_period : time := 1 ns;

  constant Ethernet_packet : std_logic_vector(511 downto 0) :=
    x"00_10_A4_7B_EA_80_00_12_34_56_78_90_08_00_45" &
    x"00_00_2E_B3_FE_00_00_80_11_05_40_C0_A8_00_2C" &
    x"C0_A8_00_04_04_00_04_00_00_1A_2D_E8_00_01_02" &
    x"03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F_10_11" &
```

```
    x"E6_C5_3D_B2";

begin

  Comp1 : fcs_check_parallel port map (
    clk            => Test_clk,
    reset          => Test_reset,
    start_of_frame => Test_start_of_frame,
    end_of_frame   => Test_end_of_frame,
    data_in        => Test_data_in,
    fcs_error      => Test_fcs_error
  );

  Clk_Generator : process
  begin
    Test_clk <= '0';
    wait for clk_period / 2;
    Test_clk <= '1';
    wait for clk_period / 2;
  end process;

  Data_Sim : process
  begin
    Test_reset <= '1';
    wait for clk_period;
    Test_reset <= '0';

    for i in 63 downto 0 loop
      if i = 63 then
        Test_start_of_frame <= '1';
      else
        Test_start_of_frame <= '0';
      end if;

      Test_data_in <= Ethernet_packet(i*8+7 downto i*8);

      if i = 3 then
        Test_end_of_frame <= '1';
      else
        Test_end_of_frame <= '0';
      end if;

      wait for clk_period;
    end loop;

    Test_data_in <= x"00";
```

```
    wait;
  end process;

end behavior;
```

Listing 4: Testbench of Parallel FCS Check

References

7 Appendix

7.1 MATLAB for Generation of CRC Value

```
% Polynomial
G = [1,0,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,0,1,1,0,1,1,1];
% Degree
degree = 32;
% Data
Data = [0x00, 0x10, 0xA4, 0x7B, 0xEA, 0x80, 0x00, 0x12, ...
        0x34, 0x56, 0x78, 0x90, 0x08, 0x00, 0x45, 0x00, ...
        0x00, 0x2E, 0xB3, 0xFE, 0x00, 0x00, 0x80, 0x11, ...
        0x05, 0x40, 0xC0, 0xA8, 0x00, 0x2C, 0xC0, 0xA8, ...
        0x00, 0x04, 0x04, 0x00, 0x04, 0x00, 0x00, 0x1A, ...
        0x2D, 0xE8, 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, ...
        0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, ...
        0x0E, 0x0F, 0x10, 0x11];
% Convert from hex to binary vector
binStr = reshape(dec2bin(Data, 8).', 1, []);
M = double(binStr) - double('0');
% Invert the first 32 bits
M(1:degree) = 1 - M(1:degree);
% Append 32 zeros for division
M = [M, zeros(1, degree)];
% XOR when the leading bit is 1 (excluding the last 32 bits)
for i = 1 : (length(M) - degree)
    if M(i) == 1
        M(i:i+degree) = xor(M(i:i+degree), G);
    end
end
% Extract and invert the last 32 bits
R = M(end-degree+1 : end);
R_final = 1 - R;
% Convert to hex and display
bit_matrix = reshape(R_final(1:32), 8, 4)';
powers_of_two = 2.^(7:-1:0)';
decimal_values = bit_matrix * powers_of_two;
hex_bytes = dec2hex(decimal_values, 2);
fprintf('Calculated Hex: 0x%s 0x%s 0x%s 0x%s\n', ...
    hex_bytes(1,:), hex_bytes(2,:), hex_bytes(3,:), hex_bytes(4,:));
```

Listing 5: MATLAB for generation of CRC value

7.2 MATLAB for Generating the Parallel Setup

```
% Parallel CRC implementation
G = [1,0,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,0,1,1,0,1,1,1];
Degree = 32;
% Construct upper-left section
v = zeros(31, 1);
I = eye(31);
upper_section_left = [v, I];
UpperLeft = [upper_section_left; fliplr(G(2:end))];
% Construct remaining matrix sections
UpperRight = zeros(32, 8);
A = zeros(1, 8); A(8) = 1;
B = zeros(7, 1); I2 = eye(7);
Bottom_right_merge = [I2, B];
BottomRight = [A; Bottom_right_merge];
BottomLeft = zeros(8, 32);
BottomLeft(1,1) = 1;
% Combine and compute M^8
Combining = [UpperLeft, UpperRight; BottomLeft, BottomRight];
M2 = mod(Combining * Combining, 2);
M4 = mod(M2 * M2, 2);
Combining8 = mod(M4 * M4, 2);
% Generate VHDL code lines
for x = 1:size(Combining8, 2)
    if x > 32, break; end
    terms = {};
    for y = 1:size(Combining8, 1)
        if Combining8(y, x) == 1
            if y > 32
                terms{end+1} = sprintf('Data_Inv(%d)', 40 - y);
            else
                terms{end+1} = sprintf('Crc_Reg(%d)', y - 1);
            end
        end
    end
    rhs = strjoin(terms, ' xor ');
    if isempty(terms), rhs = '0'; end
    fprintf('Crc_Reg(%d) <= %s;\n', x - 1, rhs);
end
```

Listing 6: MATLAB for generating the parallel setup